

Applying machine learning algorithms to predict the stock price trend in the stock market – The case of Vietnam

Giang My Tien¹, Le Thi Minh Nguyet¹, Huynh Ngoc Trang¹,
Nguyen Van Thang¹, Nguyen Thai Son¹

¹Faculty of Information Systems, University of Information Technology –
VNU-HCM, Ho Chi Minh City, Vietnam
Corresponding author: 22521464@gm.uit.edu.vn

Abstract. Forecasting stock price trends has become a critical task in modern financial analytics, offering valuable insights for investors and institutions. With the rise of machine learning (ML) and deep learning (DL) techniques, traditional forecasting methods are being enhanced by data-driven approaches capable of capturing non-linear and temporal dependencies in stock data. This paper investigates the effectiveness of various machine learning and deep learning models in predicting the stock prices of Vietcombank (VCB), one of the largest and most influential commercial banks in Vietnam - using historical data from 2020 to 2024 collected via investing.com. We conduct a comprehensive evaluation of ten models, including classical approaches (Linear Regression, SARIMA, SARIMAX), machine learning methods (Support Vector Regression, LightGBM), and deep learning architectures (RNN, LSTM, GRU, CNN+LSTM, CNN+Transformer). Performance is assessed using MAE, RMSE, and MAPE, with a focus on MAPE for robust comparison. Among all models, Linear Regression delivered the most accurate forecasts with a MAPE of 0.30%, followed by GRU (0.65%) and SARIMAX (0.66%). These results highlight the potential of recurrent-based models for financial time series forecasting in emerging markets like Vietnam.

Keywords: stock price forecasting, time series analysis, machine learning, deep learning, Vietnam stock market, Linear Regression, SARIMA, SARIMAX, SVR, LightGBM, RNN, LSTM, GRU, CNN-LSTM, CNN-Transformer.

1 Introduction

The stock market serves as a critical pillar in Vietnam's economic growth, offering a platform for capital mobilization, investment diversification, and financial risk management. For enterprises, it enables raising long-term capital through equity issuance, while for investors, it opens access to diverse asset classes with potential returns. In recent years, Vietnam's stock market has grown rapidly in both scale and complexity, attracting increasing participation from domestic and international investors [1].

However, forecasting stock prices remains a complex task due to their inherently volatile, nonlinear, and dynamic nature. Traditional time series models such as ARIMA or SARIMA often fall short in capturing abrupt market shifts and long-term dependencies. As a result, the application of machine learning (ML) and deep learning (DL) models has gained popularity in financial forecasting, owing to their ability to learn hidden patterns and nonlinear relationships from historical data [2][3].

This paper investigates the predictive performance of ten models - including classical (Linear Regression, SARIMA, SARIMAX), ML - based (SVR, LightGBM), and DL-based architectures (RNN, LSTM, GRU, CNN+LSTM, CNN+Transformer) - in forecasting the stock prices of Vietcombank (VCB) using daily data from 2020 to 2024. These banks represent key players in the Vietnamese financial system and serve as proxies for market behavior.

The goal is to evaluate these models under consistent preprocessing and experimental conditions, identifying the most accurate approach based on metrics such as MAE, RMSE, and MAPE. Moreover, the best-performing model for each company will be used to forecast stock prices for the next 30 days. This comparative analysis not only highlights the relative strengths of each modeling technique but also contributes to ongoing research on stock price prediction in emerging markets and supports data-driven investment strategies in Vietnam.

2 Related Works

Stock price prediction has been extensively studied using a wide range of statistical and machine learning methods. Linear Regression remains one of the most fundamental approaches in forecasting. James et al. (2013) [4] provide a comprehensive introduction to statistical learning, including linear models, highlighting their simplicity and interpretability in regression tasks.

To model seasonal and autoregressive trends in financial data, SARIMA and SARIMAX have been widely applied. Hyndman and Athanasopoulos (2021) [5][6] offer practical guidelines for implementing these models, while Box et al. (2015) [7][8] provide the theoretical foundation for time series analysis and control in financial forecasting.

In the domain of machine learning, Support Vector Regression (SVR) has shown effectiveness in capturing non-linear relationships in financial datasets. Smola and Schölkopf (2004) [9] present a detailed tutorial on SVR, outlining its theoretical framework and advantages in regression tasks. Similarly, LightGBM, a gradient boosting framework introduced by Ke et al. (2017) [10], has gained popularity due to its speed and high accuracy on structured data.

Recurrent Neural Networks (RNNs) have marked a significant advancement in modeling sequential financial data. Elman (1990) [11] introduced the basic RNN architecture, while Hochreiter and Schmidhuber (1997) [12] developed the Long Short-Term Memory (LSTM) model to address the vanishing gradient problem in long-term sequences. Cho et al. (2014) [13] later proposed the Gated Recurrent Unit (GRU) as a simplified and efficient alternative to LSTM.

Beyond standard recurrent models, hybrid architectures have also been explored. Donahue et al. (2015) [14] introduced the CNN-LSTM model, combining convolutional and recurrent layers to extract both spatial and temporal features. Recently, Zeng et al. (2023) [15] applied a CNN-Transformer architecture for financial time series forecasting, demonstrating the potential of attention-based models in capturing complex dependencies in stock data.

These studies collectively provide a strong foundation for evaluating and comparing various models in stock price prediction, particularly in the context of emerging markets like Vietnam.

3 Method

3.1 Data Overview

Data Collection and Preparation

Based on reputable technology sources and financial analysis articles, we identified a reliable data source from vn.investing.com, which provides historical financial data across various sectors including gold, silver, and stock prices. In this study, we collected historical stock data for the Joint Stock Commercial Bank for Foreign Trade of Vietnam (Vietcombank – VCB) from January 1, 2020, to December 31, 2024. The dataset was downloaded in CSV format and originally contained seven columns in Vietnamese: “Ngày” (Date), “Lần cuối” (Last), “Mở” (Open), “Cao” (High), “Thấp” (Low), “KL” (Volume), and “% Thay đổi” (Percentage Change).

Data Visualization

Before proceeding to data preprocessing, it is crucial to gain an initial understanding of the data through exploratory data analysis (EDA). A summary of descriptive statistics for the price-related attributes of VCB stock is shown in Table 1. Subsequently, *Close* are visualized to detect potential outliers or patterns over time (Figure 1).

Table 1. Data Summary about VCB Dataset

	Close	Open	High	Low	Volume
Count	1221	1221	1221	1221	1221
Mean	7.714.125	7.712.006	7.794.286	7.630.217	1447554
std	1.129.137	1.133.449	1.129.006	113.579	7.483.639
min	43925	43925	44847	43465	266550
25%	67194	67231	68163	66554	943380
50%	77053	77053	77985	76103	1300000
75%	87500	87400	88300	86622	1800000

max	97400	98500	100500	97300	8010000
median	77053	77053	77985	76103	1300000
mode	92000	92000	92000	92000	92000
range	53475	54575	55653	53835	7743450

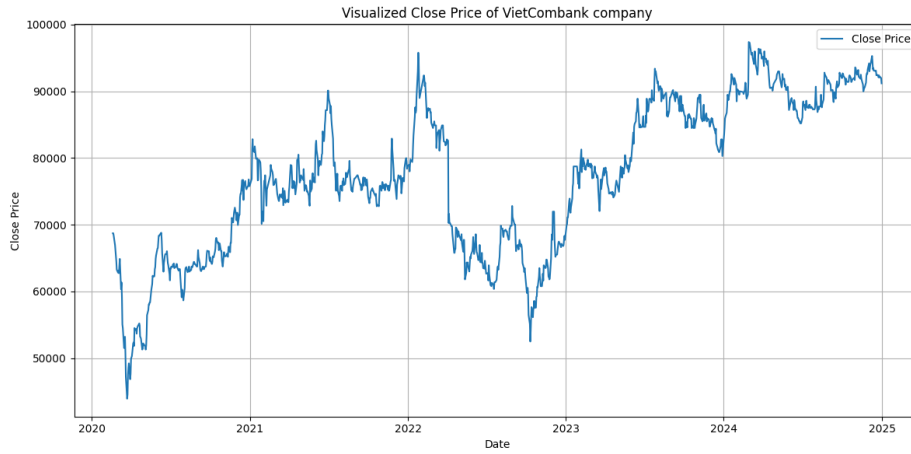


Fig. 1. Visualized Close price of VCB Dataset

Data Preprocessing

The dataset initially contained non-standardized formats, necessarily to preprocessing before model development. All column headers were translated from Vietnamese to English to ensure consistency and compatibility with machine learning workflows: specifically, “Ngày” was renamed to Date, “Lần cuối” to Close, “Mở” to Open, “Cao” to High, “Thấp” to Low, “KL” to Volume, and “% Thay đổi” to Pct_Change. The Volume column included unit suffixes such as “K” (thousands) and “M” (millions), which were programmatically converted to unified numerical formats. All columns were then cast to appropriate data types, with the Date column converted to datetime and all other variables converted to float to ensure mathematical operability. Integrated within this preprocessing pipeline, we performed essential feature engineering to enhance the model's predictive capacity. This included constructing lag features (e.g., previous-day prices), moving averages (e.g., 7-day and 30-day rolling means), volatility measures (e.g., rolling standard deviations), and percentage changes over various time intervals. These derived features are particularly valuable in capturing temporal dependencies and underlying market trends, thereby improving the performance of time series forecasting models.

Data Normalization

Data normalization (or feature scaling) plays a crucial role in optimizing model convergence, especially for distance-based or neural network models. Two widely adopted normalization techniques were considered:

1. StandardScaler (Z-score normalization)

$$z = \frac{x - \mu}{\sigma}$$

where μ is the mean and σ is the standard deviation.

2. MinMaxScaler

$$x' = \frac{x - x_{\min}}{x_{\max} - x_{\min}}$$

The choice of scaling technique depends on the model architecture. While linear regression may perform adequately without scaling, more sensitive models such as Support Vector Machines (SVMs), K-Nearest Neighbors (KNN), and neural networks often benefit significantly from normalization.

Data Splitting

After preprocessing and normalization, the dataset was partitioned into three subsets for model training, validation, and testing. This stratification enables a more robust evaluation of model performance and mitigates overfitting. The following data split ratios were applied for comparative analysis:

- 65% training / 25% validation / 10% testing
- 70% training / 20% validation / 10% testing
- 75% training / 15% validation / 10% testing

Each configuration allows us to assess the impact of training size on generalization performance and model consistency.

Feature Selection

To reduce dimensionality and improve model performance, we applied Pearson correlation analysis to evaluate the linear relationship between each feature and the target variable Close. As shown in Figure 2, we selected features with relatively strong correlation (coefficient ≥ 0.75), specifically those from Year and above in the heatmap. Features with low correlation were excluded to avoid noise and overfitting, ensuring a more efficient and generalizable model.

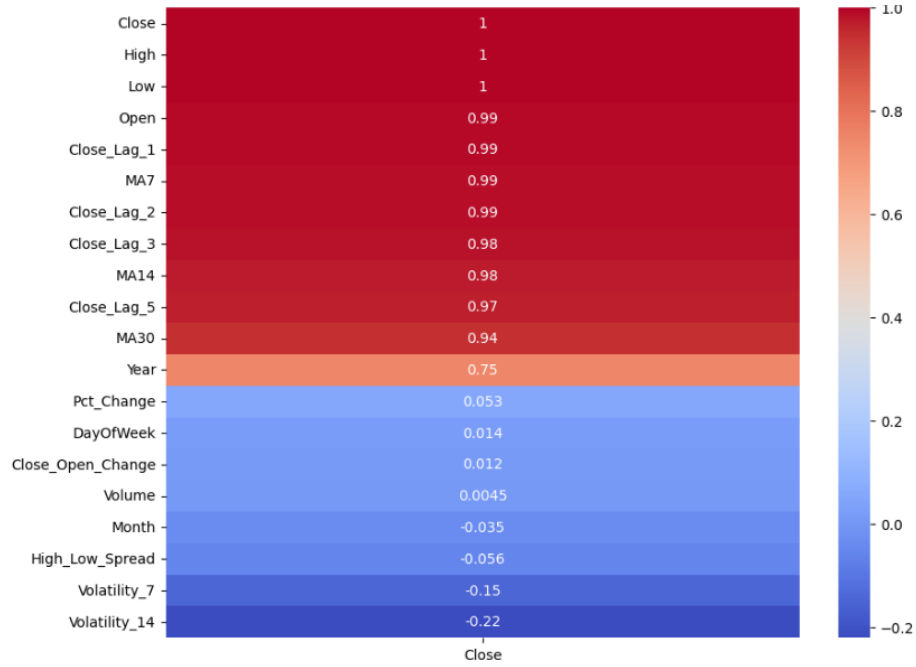


Fig. 2. Pearson correlation heatmap between features and the Close price

3.2 Model

Linear Regression.

Linear Regression is one of the simplest and most widely used predictive models in statistics and machine learning. It models the relationship between a dependent variable y and one or more independent variables X by fitting a linear equation [4]:

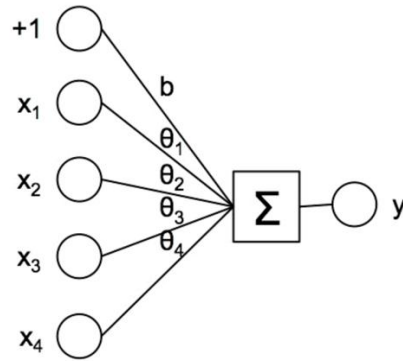


Fig. 3. Linear Regression Arrchitecture

$$y = X\beta + \epsilon$$

Where:

- y : Target (dependent) variable.
- X : Input features (independent variables).
- β : Coefficients (weights) to be learned.
- ϵ : Error term (residual).

The model learns β by minimizing the Mean Squared Error (MSE) between the predicted values $\hat{y}$ and the actual values y [16]:

$$MSE = \frac{1}{n} \sum_{i=1}^n (y_i - \hat{y}_i)^2$$

Linear regression assumes that there is a linear relationship between the inputs and the target, the errors are normally distributed, and the variance of the errors is constant (homoscedasticity).

Despite its simplicity, linear regression serves as a strong baseline in many tasks and provides interpretable coefficients, making it a valuable tool for understanding relationships in the data. It is commonly applied in forecasting, risk modeling, and as a benchmark in time series prediction tasks [17].

SARIMA.

Seasonal Autoregressive Integrated Moving Average (SARIMA) is an advanced time series forecasting model that extends ARIMA by incorporating seasonal components. While ARIMA (Autoregressive Integrated Moving Average) handles non-seasonal data by modeling linear relationships in lagged observations and errors, SARIMA adapts this approach for data with recurring seasonal patterns, making it well-suited for real-world phenomena like temperature, sales, or traffic flow that repeat over regular intervals [5].

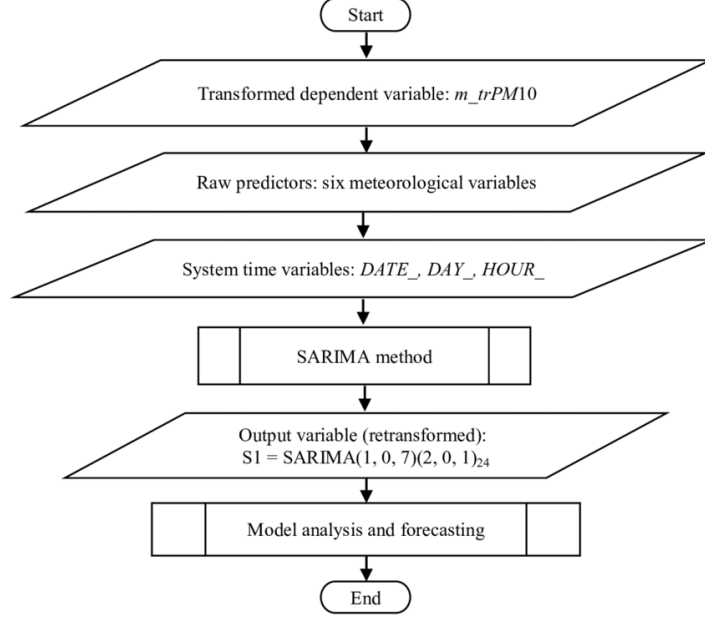


Fig. 4. SARIMA Model Structure

SARIMA enhances ARIMA by adding seasonal autoregressive (SAR), seasonal differencing (D), and seasonal moving average (SMA) terms. The full model is usually denoted as:

$$\text{SARIMA}(p, d, q)(P, D, Q)_s$$

Where:

- p, d, q : Non-seasonal autoregressive order, differencing order, and moving average order.
- P, D, Q : Seasonal autoregressive, differencing, and moving average order.
- s : Length of the seasonal cycle (e.g., 12 for monthly data with yearly seasonality).

Model Equation (conceptually):

$$\Phi_P(B^s)\phi_P(B)\nabla^d\nabla_s^D y_t = \Theta_Q(B^s)\theta_q(B)\epsilon_t$$

Where:

- B : Backshift operator (i.e., $By_t = y_{t-1}$)
- $\phi_P(B), \theta_q(B)$: Polynomials for AR and MA components.
- $\Phi_P(B^s), \Theta_Q(B^s)$: Seasonal AR and MA polynomials.
- ∇^d : Non-seasonal differencing operator.
- $\nabla_s^D = (1 - B^s)^D$: Seasonal differencing operator.
- ϵ_t : White noise error.

SARIMA is highly interpretable and widely used in traditional time series forecasting tasks. However, it assumes stationarity after differencing and struggles with nonlinear or long-range temporal dependencies, which modern deep learning models like LSTM or Transformer may better capture [7][18].

SARIMAX.

SARIMAX is an extension of the SARIMA model that allows the inclusion of external (exogenous) variables to improve time series forecasting. While SARIMA models only the internal dynamics of a time series (i.e., its autoregressive, differencing, moving average, and seasonal components), SARIMAX incorporates additional predictors that may influence the target variable, such as economic indicators, marketing campaigns, or weather data [6][8].

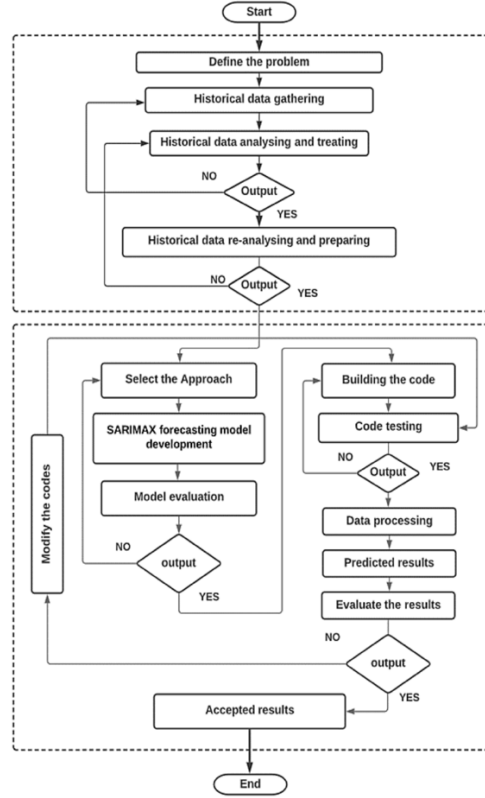


Fig. 5. SARIMAX Model Structure

The general SARIMAX model is denoted as:

$$\text{SARIMAX}(p, d, q)(P, D, Q)_s$$

with additional exogenous variables X_t , where:

- p, d, q : Non-seasonal autoregressive order, differencing order, and moving average order.
- P, D, Q : Seasonal autoregressive, differencing, and moving average order.
- s : Seasonal period.
- X_t : Vector of exogenous regressors at time t .

Model Equation (conceptually):

$$y_t = \mu + \phi_1 y_{t-1} + \dots + \phi_p y_{t-p} + \theta_1 \epsilon_{t-1} + \dots + \theta_q \epsilon_{t-q} + \beta X_t + \epsilon_t$$

Where:

- ϕ : Autoregressive coefficients.
- θ : Moving average coefficients.
- β : Coefficients for exogenous variables.
- ϵ_t : White noise error.

The seasonal components (SAR, SMA) follow similar patterns as in SARIMA, but the key advantage of SARIMAX is its ability to explicitly account for external influences, improving forecast accuracy when such external drivers are present [19][20].

SVR.

Support Vector Regression (SVR) is a type of Support Vector Machine (SVM) adapted for regression tasks. Unlike classification-focused SVMs, SVR is designed to predict continuous values by finding a function that deviates from the actual observed values by a margin no greater than ϵ , while being as flat as possible. This makes SVR particularly effective in applications such as time series forecasting, financial prediction, and system modeling, where capturing complex relationships between input features and continuous outputs is essential [9].

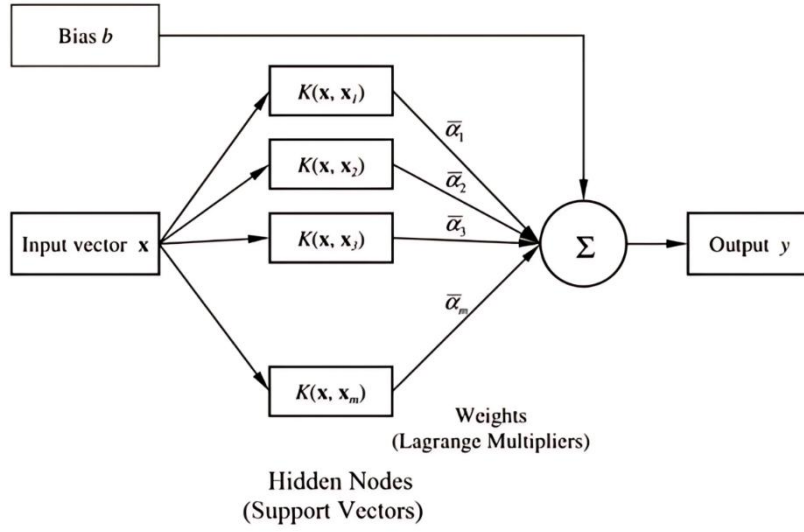


Fig. 6. SVR Architecture

The key idea of SVR is to map the input data into a high-dimensional feature space using a kernel function, and then to construct a linear function in that space. This function must predict within a margin of tolerance ϵ from the true values, and only data points outside this margin (called support vectors) influence the model.

Given a training dataset $\{(x_i, y_i)\}_{i=1}^n$, the SVR optimization problem is defined as:

$$\min_{w, b, \xi, \xi^*} \frac{1}{2} \|w\|^2 + C \sum_{i=1}^n (\xi_i + \xi_i^*)$$

Subject to:

$$\begin{cases} y_i - \langle w, \varphi(x_i) \rangle - b \leq \varepsilon + \xi_i \\ \langle w, \varphi(x_i) \rangle + b - y_i \leq \varepsilon + \xi_i^* \\ \xi_i, \xi_i^* \geq 0 \end{cases}$$

Where:

- w : Weight vector in the high-dimensional space.
- b : Bias term.
- $\varphi(x)$: Mapping function to a higher-dimensional space.
- ε : The margin of tolerance for error.
- ξ_i, ξ_i^* : Slack variables representing deviations from the margin.
- C : Regularization parameter controlling the trade-off between the flatness of the function and the tolerance to deviations beyond ε .

The prediction function is defined as:

$$f(x) = \sum_{i=1}^n (\alpha_i - \alpha_i^*) K(x_i, x) + b$$

Where:

- α_i, α_i^* : Lagrange multipliers determined during training.
- $K(x_i, x)$: Kernel function (e.g., linear, polynomial, RBF).

SVR is powerful in modeling nonlinear relationships by using kernels to implicitly transform data into a space where a linear model is effective. However, its performance is highly sensitive to hyperparameters like C , ε , and the choice of kernel, which require careful tuning [21].

Despite these challenges, SVR remains a popular choice for regression tasks, especially when interpretability and robustness to outliers are desired.

LightGBM.

LightGBM (Light Gradient Boosting Machine) is a gradient boosting framework developed by Microsoft that uses tree-based learning algorithms. It is designed for distributed and efficient training, providing faster training speed and higher efficiency compared to traditional gradient boosting techniques. LightGBM is well-suited for large datasets and has become widely used in data science competitions and real-world applications [10].

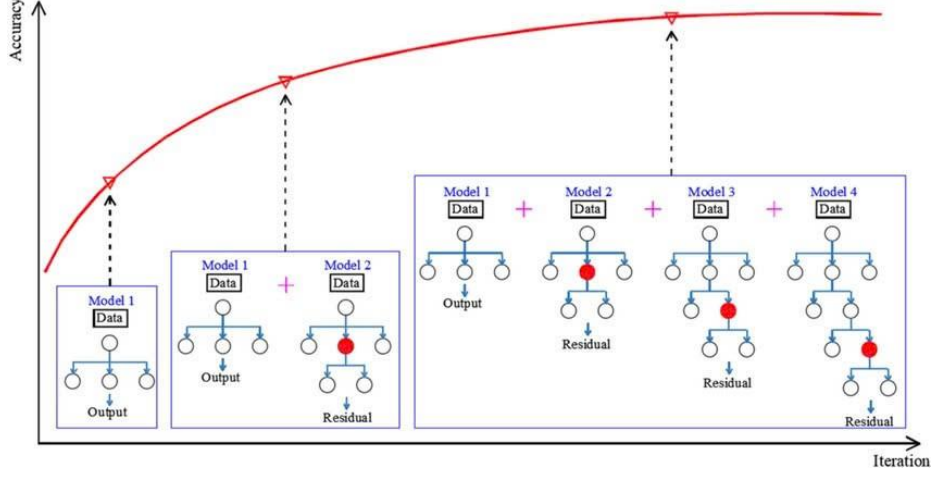


Fig. 7. LightGBM Architecture

The key idea behind LightGBM is to grow decision trees leaf-wise (best-first) rather than level-wise. This approach tends to reduce more loss compared to level-wise growth, leading to better accuracy. In addition, LightGBM uses histogram-based algorithms to speed up the training process and reduce memory usage.

The core steps of LightGBM training are as follows:

1. Bucket continuous feature values into discrete bins (histograms).
2. For each iteration, find the leaf with the highest potential gain and split it.
3. Continue splitting until the maximum depth or minimum data in leaf constraint is met.

LightGBM minimizes a loss function L by adding weak learners (trees) in a sequential manner. At iteration t , a new tree is fitted to the negative gradients (also called pseudo-residuals) of the loss function with respect to the current model predictions:

$$g_i = \frac{\partial L(y_i, \hat{y}_t)}{\partial \hat{y}_t}$$

$$h_i = \frac{\partial^2 L(y_i, \hat{y}_t)}{\partial \hat{y}_t^2}$$

Then, a new tree $h_t(x)$ is trained to minimize the following objective:

$$Obj = \sum_i \left[g_i h_t(x_i) + \frac{1}{2} h_i h_t(x_i)^2 \right] + \Omega(h_t)$$

Where:

- g_i : First-order gradient (loss derivative).
- h_i : Second-order gradient (Hessian).
- $h_i(x)$: The new decision tree at iteration t .
- $\Omega(h_i)$: Regularization term to control the complexity of the model.

Advantages of LightGBM include:

- Faster training and lower memory usage due to histogram-based approach.
- Support for categorical features directly without one-hot encoding.

- Better accuracy by leaf-wise tree growth.
- Scalability to large datasets and distributed environments.

Despite its advantages, LightGBM can be prone to overfitting if not properly tuned. Proper control of parameters such as `max_depth`, `min_data_in_leaf`, and `learning_rate` is important for stable performance [22].

RNN.

Recurrent Neural Networks (RNNs) are a class of neural networks designed to process sequential data by maintaining a form of memory through hidden states. Unlike feed-forward networks, RNNs have loops in their architecture, allowing information to persist over time. This capability makes them suitable for tasks such as time series prediction, speech recognition, and natural language processing where temporal or sequential relationships are important [11].

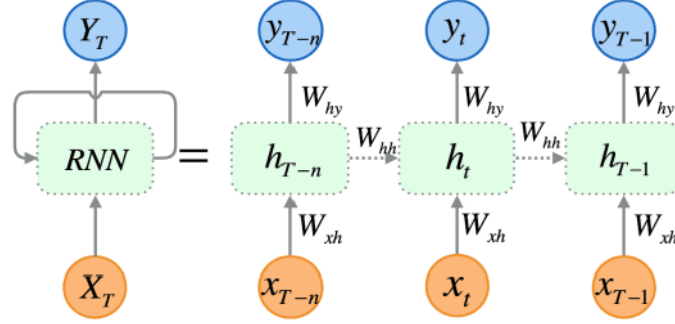


Fig. 8. RNN Architecture

The key idea of RNN is to use the same set of weights at each time step while maintaining a hidden state that captures past information. At time step t , the network receives an input vector x_t , updates its hidden state h_t , and generates an output y_t .

The computations are defined as:

$$h_t = f(W_{xh}x_t + W_{hh}h_{t-1} + b_h)$$

Where:

- h_t : Hidden state at time t .
- x_t : Input at time t .
- W_{xh} : Weight matrix connecting the input to the hidden state.
- W_{hh} : Weight matrix connecting the previous hidden state to the current hidden state.
- b_h : Bias term for the hidden state.
- f : Activation function (commonly tanh or ReLU).

$$y_t = W_{hy}h_t + b_y$$

Where:

- y_t : Output at time t .
- W_{hy} : Weight matrix connecting the hidden state to the output.
- b_y : Bias term for the output.

RNNs can theoretically learn long-term dependencies in data; however, in practice, they struggle with the vanishing and exploding gradient problems. This makes it difficult to retain relevant information over long sequences during training, limiting their effectiveness on tasks that require learning dependencies across many time steps [23]. Despite this limitation, RNNs are the foundational model for sequential learning and have inspired more advanced architectures like LSTM and GRU, which directly address these challenges.

LSTM.

Long Short-Term Memory (LSTM) networks are an enhanced type of RNN specifically designed to overcome the vanishing gradient problem and to learn long-term dependencies in sequential data. Proposed by Hochreiter and Schmidhuber in 1997, the LSTM introduces a memory cell and gating mechanisms that regulate the flow of information [12].

An LSTM unit contains three main gates: the forget gate, the input gate, and the output gate. These gates control how much of the previous information is kept, how much new information is added, and how much of the internal state is exposed, respectively.

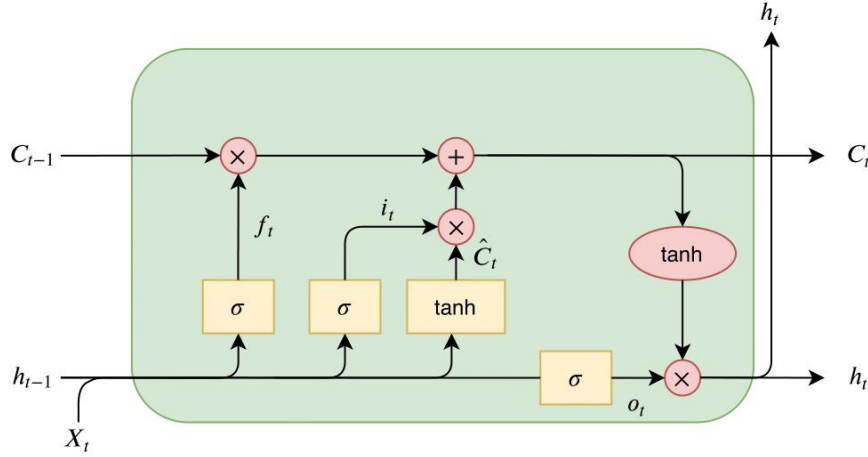


Fig. 9. LSTM architecture

Mathematically, the forward pass of an LSTM cell at time t is governed by the following equations:

Forget Gate: Decides what information to discard from the cell state.

$$f_t = \sigma(W_f \cdot [h_{t-1}, x_t] + b_f)$$

Input Gate: Determines what new information to store in the cell.

$$i_t = \sigma(W_i \cdot [h_{t-1}, x_t] + b_i)$$

Candidate Cell State: Creates a vector of new candidate values $\tilde{C}_t$, which could be added to the state.

$$\tilde{C}_t = \tanh(W_C \cdot [h_{t-1}, x_t] + b_C)$$

Update Cell State: Combines the previous cell state and the new candidate values, controlled by the forget and input gates.

$$C_t = i_t * \tilde{C}_t + f_t * C_{t-1}$$

Output Gate: Determines the output based on the current cell state.

$$o_t = \sigma(W_o \cdot [h_{t-1}, x_t] + b_o)$$

Hidden State Update: Computes the new hidden state to be passed to the next time step.

$$h_t = o_t * \tanh(C_t)$$

Here, σ denotes the sigmoid activation, $\tanh$ is the hyperbolic tangent function, and $*$ is the element-wise multiplication. C_t is the cell state that carries long-term information, and h_t is the hidden state exposed at each time step.

The memory cell C_t enables the LSTM to carry information across long sequences, which allows the model to learn complex temporal dynamics. Empirical studies have shown that LSTMs outperform traditional RNNs in tasks such as speech recognition, machine translation, and time series forecasting [24].

Additionally, LSTMs are robust to input noise and can generalize well on unseen sequential patterns, making them a strong choice for sequence modeling tasks.

GRU.

The Gated Recurrent Unit (GRU) is a more recent development introduced by Cho et al. (2014) as a simplified version of the LSTM. GRUs combine the forget and input gates into a single update gate and merge the cell and hidden states, which reduces the model complexity while retaining its ability to learn long-term dependencies [13].

GRUs consist of two main gates: the update gate z_t and the reset gate r_t . These gates control the flow of information similarly to LSTM but with fewer operations.

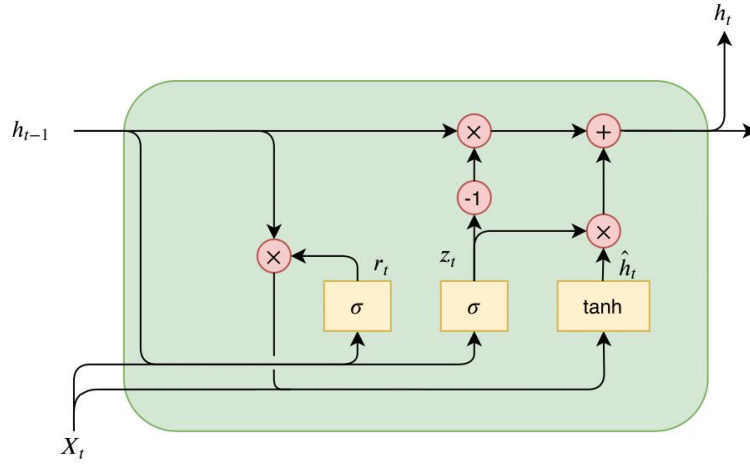


Fig. 10. GRU architecture

The equations governing a GRU at time step t are:

Update gate:

$$z_t = \sigma(W_z \cdot [h_{t-1}, x_t])$$

Reset gate:

$$r_t = \sigma(W_r \cdot [h_{t-1}, x_t])$$

Candidate hidden state:

$$\tilde{h}_t = \tanh(W_h \cdot [r_t * h_{t-1}, x_t])$$

Final hidden state:

$$h_t = (1 - z_t) * h_{t-1} + z_t * \tilde{h}_t$$

In this formulation, the update gate z_t determines how much of the previous state is retained, while the reset gate r_t controls the influence of the past state in the computation of the candidate hidden state $\tilde{h}_t$.

The GRU architecture has been shown to perform comparably to LSTM on many tasks, despite having fewer parameters. This makes it more computationally efficient and faster to train, especially for large-scale problems or limited computational resources [25].

Moreover, GRUs are often favored when the dataset is smaller or when model interpretability and speed are more critical than raw accuracy.

CNN + LSTM.

Convolutional Neural Network combined with Long Short-Term Memory (CNN + LSTM) is a hybrid deep learning architecture designed to capture both spatial and temporal features in data. CNNs are effective at extracting local patterns from input data such as images or short time windows, while LSTMs are designed to model temporal dependencies by maintaining long-term memory across sequences. This combination makes CNN + LSTM models particularly useful for tasks involving spatiotemporal data such as video classification, activity recognition, and multivariate time series forecasting [14].

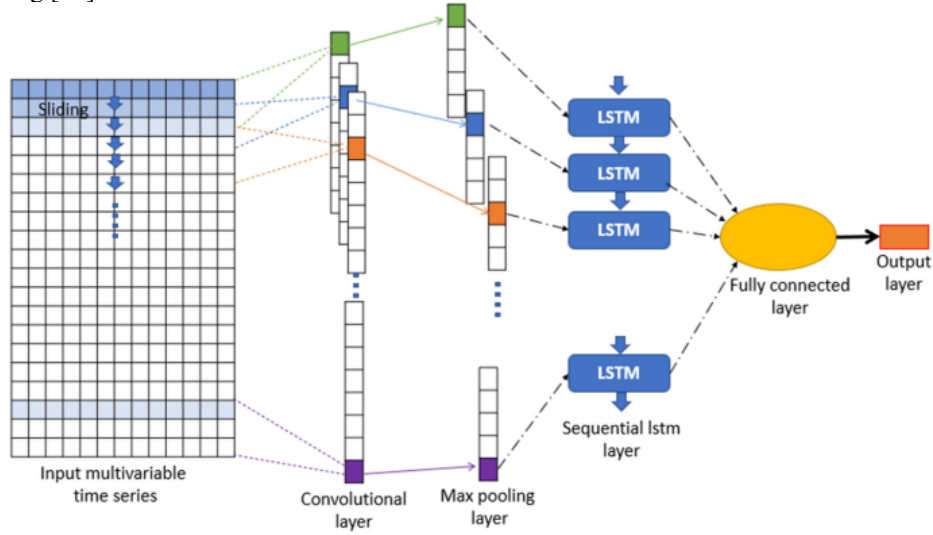


Fig. 11. CNN + LSTM Architecture

The key idea of CNN + LSTM is to first apply convolutional layers to extract high-level spatial features from the input, and then feed the resulting feature sequence into an LSTM network to learn temporal relationships over time [26].

The computations typically follow these steps:

1. CNN-based feature extraction:

Input data (e.g., frames or windowed time series) are passed through convolutional layers. A typical convolution operation is defined as:

$$F_{i,j}^k = \sigma\left(\sum_{m,n} K_{m,n}^{(k)} \cdot X_{i+m,j+n} + b^{(k)}\right)$$

Where:

- $F_{i,j}^k$: Output feature at location (i,j) in the kkk-th filter.
- $K^{(k)}$: Convolutional kernel/filter.
- X : Input patch.
- $b^{(k)}$: Bias term.
- σ : Activation function (e.g., ReLU).

2. LSTM-based sequence modeling:

The extracted features (now in sequence form) are passed to the LSTM network. At time step t , the LSTM updates are:

$$\begin{aligned} i_t &= \sigma(W_{xi}x_t + W_{hi}h_{t-1} + b_i) \\ f_t &= \sigma(W_{xf}x_t + W_{hf}h_{t-1} + b_f) \\ o_t &= \sigma(W_{xo}x_t + W_{ho}h_{t-1} + b_o) \\ c_t &= f_t \odot c_{t-1} i_t + \tanh \odot (W_{xc}x_t + W_{hc}h_{t-1} + b_c) \\ h_t &= o_t \odot \tanh(c_t) \end{aligned}$$

3. Output layer:

The final hidden state h_t or a sequence of hidden states is passed through dense layers for classification or prediction.

This hybrid architecture enables the model to learn complex dependencies in data where both structure and order matter [27].

CNN + Transformer.

Convolutional Neural Network combined with Transformer (CNN + Transformer) is a hybrid architecture that leverages both local feature extraction and global context modeling. CNNs are highly effective at capturing local patterns such as edges, textures, and short-range dependencies through convolutional layers, while Transformers are designed to model long-range dependencies and global relationships using self-attention mechanisms.

This combination has been widely applied in fields such as image recognition, video analysis, and time series forecasting, where both local details and global context matter. By first extracting feature maps with CNN layers and then feeding them into a Transformer encoder or decoder, the model can integrate both spatial and temporal information effectively [15].

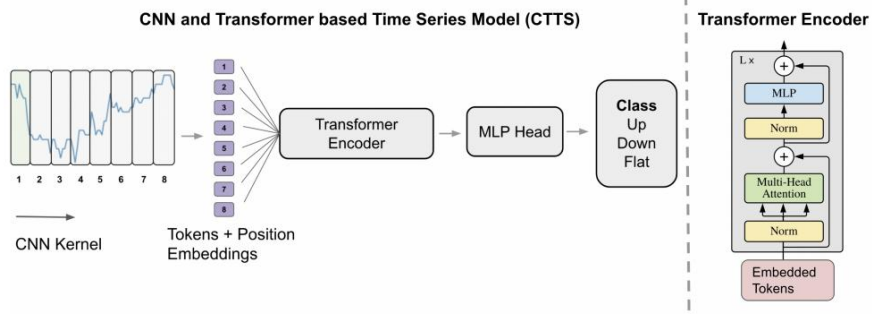


Fig. 12. CNN + Transformer Architecture

1. CNN Layer:

Input data (e.g., frames or windowed time series) are passed through convolutional layers. A typical convolution operation is defined as:

$$F_{i,j}^k = \sigma \left(\sum_{m,n} K_{m,n}^{(k)} \cdot X_{i+m,j+n} + b^{(k)} \right)$$

Where:

- $F_{i,j}^k$: Output feature at location (i,j) in the kkk-th filter.
- $K^{(k)}$: Convolutional kernel/filter.
- X : Input patch.
- $b^{(k)}$: Bias term.
- σ : Activation function (e.g., ReLU).

2. Transformer Encoder Layer:

The Transformer follows this overall architecture using stacked self-attention and point-wise, fully connected layers for both the encoder and decoder, shown in the left and right halves of Figure 1, respectively [28].

$$Q = XW^Q, K = XW^K, V = XW^V$$

$$Attention(Q, K, V) = softmax \left(\frac{QK^T}{\sqrt{d_k}} \right) V$$

Where:

- Q, K, V : Query, Key, Value matrices.
- W^Q, W^K, W^V : Learnable weight matrices.
- d_k : Dimension of the key vector (used for scaling).
- $softmax$: Converts attention scores into a probability distribution.

After the self-attention layer, the output goes through:

- Add & Norm (residual connection + layer normalization).
- Feedforward network (FFN):

$$FFN(x) = \max(0, xW_1 + b_1) W_2 + b_2$$

3.3 Evaluation Methodology

Mean Absolute Percentage Error (MAPE)

MAPE is the mean absolute percentage error, which is a relative measure that essentially scales MAD to be in percentage units instead of the variable's units. Mean absolute percentage error is a relative error measure that uses absolute values to keep the positive and negative errors from canceling one another out and uses relative errors to enable you to compare forecast accuracy between time-series models [29].

Formula:

$$MAPE = \frac{1}{n} \sum_{t=1}^n \left| \frac{A_t - F_t}{A_t} \right|$$

Where:

- n : the number of fitted points
- A_t : the actual value
- F_t : the forecast value

Root Mean Square Error (RMSE)

Root Mean Square Error (RMSE) is the standard deviation of the residuals (prediction errors). Residuals are a measure of how far from the regression line data points are; RMSE is a measure of how spread out these residuals are. In other words, it tells you how concentrated the data is around the line of best fit [30].

Formula:

$$RMSE = \sqrt{\overline{(f - o)^2}}$$

Where:

- f : forecasts (expected values or unknown results)
- o : observed values (known results)

Mean Absolute Error (MAE)

MAE is the mean absolute error, a metric that calculates the average of the absolute differences between predicted values and actual values. Unlike MAPE, MAE is a scale-dependent metric expressed in the same units as the data. It uses absolute values to prevent positive and negative errors from canceling each other out, providing a straightforward measure of forecast accuracy [31].

Formula:

$$MAE = \frac{1}{n} \sum_{t=1}^n |A_t - F_t|$$

Where:

- n : the number of data points
- A_t : the actual value at time t
- F_t : the forecast value at time t

4 Results

Table 2. Evaluation of Forecasting Models (Train/Val/Test = 65/25/10)

Model	MAE	MAPE (%)	RMSE
Linear Regression	272.77	0.30	361.06
GRU	595.93	0.65	756.52
RNN	667.91	0.73	841.17
CNN + Transformer	912.38	1.00	1121.70
SARIMAX	1046.22	1.15	1120.74
LightGBM	1456.54	1.59	2267.41
CNN + LSTM	2129.39	2.33	2278.34
LSTM	6103.14	6.62	6201.80
SVR	10630.41	11.56	12187.04
SARIMA	16482.12	18.07	16606.57

Table 3. Evaluation of Forecasting Models (Train/Val/Test = 70/20/10)

Model	MAE	MAPE (%)	RMSE
Linear Regression	283.61	0.31	368.65
SARIMAX	596.81	0.66	671.33
GRU	758.85	0.83	874.91
RNN	1027.69	1.11	1185.23
LightGBM	1416.57	1.54	2174.14
LSTM	1604.58	1.73	1844.24
CNN + LSTM	2236.12	2.45	2372.97
SARIMA	2688.32	2.92	3037.92
CNN + Transformer	4100.75	4.48	4392.61
SVR	5606.85	6.13	6413.81

Table 4. Evaluation of Forecasting Models (Train/Val/Test = 75/15/10)

Model	MAE	MAPE (%)	RMSE
Linear Regression	293.20	0.32	376.00
GRU	635.57	0.69	835.66
LSTM	717.37	0.78	913.34
CNN + Transformer	726.91	0.80	912.83
RNN	780.33	0.85	997.49
SARIMAX	794.33	0.87	864.28
LightGBM	1081.46	1.16	1789.27
SVR	1470.11	1.60	1884.04
CNN + LSTM	2236.12	2.45	2372.97
SARIMA	5035.61	5.49	5429.10

Tables 2, 3, and 4 present the performance of various forecasting models across three different data split scenarios: Train/Validation/Test ratios of 65/25/10, 70/20/10, and 75/15/10, respectively. The evaluation metrics include Mean Absolute Error (MAE), Mean Absolute Percentage Error (MAPE), and Root Mean Squared Error (RMSE).

Across all splits, Linear Regression consistently achieves the lowest MAPE values (approximately 0.30–0.32%), demonstrating surprisingly strong performance despite its simplicity. Notably, in the 65/25/10 setting (Table 2), it yields a MAPE of 0.30%, outperforming more complex deep learning models. This suggests that, for this dataset, linear trends capture much of the predictive signal.

Among recurrent neural networks, GRU consistently shows stable and competitive performance, with MAPE ranging from 0.65% to 0.83% and relatively low MAE and RMSE values across all tables. Compared to RNN and LSTM, GRU often achieves lower errors, indicating a better ability to model temporal dependencies in the VCB stock dataset. In contrast, LSTM and RNN models tend to show higher variance and weaker performance. For example, LSTM under the 65/25/10 split reaches a MAPE of 6.62% and RMSE above 6000, likely due to sensitivity to training data size or overfitting.

Hybrid models such as CNN + Transformer and CNN + LSTM do not consistently outperform simpler alternatives. The CNN + Transformer model demonstrates substantial fluctuation in performance—achieving a relatively low MAPE of 0.80% (Table 4) but rising to 4.48% under the 70/20/10 split—suggesting a high sensitivity to data partitioning and potential instability.

Traditional time series approaches, including SARIMAX and SARIMA, show mixed results. SARIMAX performs competitively under the 75/15/10 split with a MAPE of 0.87%, but its effectiveness diminishes in other settings. SARIMA, meanwhile, consistently underperforms with MAPE exceeding 5% in some configurations, reflecting its limitations in handling complex temporal dynamics compared to more modern approaches.

Support Vector Regression (SVR) emerges as the least stable model, with performance deteriorating significantly across all splits. Its MAPE reaches as high as 11.56% in Table 2 and remains suboptimal in the other scenarios, confirming that SVR is not well suited for this particular forecasting task.

Overall, the choice of train/validation/test ratio clearly impacts model performance. Larger training sets (e.g., 75/15/10) generally improve the accuracy of deep learning models by providing more data to learn from. However, some models—particularly LSTM and SVR—exhibit inconsistent or degraded results across different splits, highlighting sensitivity to training size and possible overfitting. In contrast, simpler models like Linear Regression display robust generalization across all configurations, suggesting their reliability in scenarios with limited data or high volatility. Thus, both the model selection and data partitioning strategy play crucial roles in determining forecasting effectiveness.

These results also suggest that the VCB stock dataset exhibits relatively strong linear patterns and low short-term volatility, making it favorable for simpler statistical models and moderately deep architectures like GRU.

For each model, the best-performing MAPE ratio is reported, accompanied by relevant visualizations to support comparative analysis.



Fig. 13. Linear Regression Forecast with a 65/25/10 Data Split



Fig. 14. SARIMA Forecast with a 70/20/10 Data Split



Fig. 15. SARIMAX Forecast with a 70/20/10 Data Split

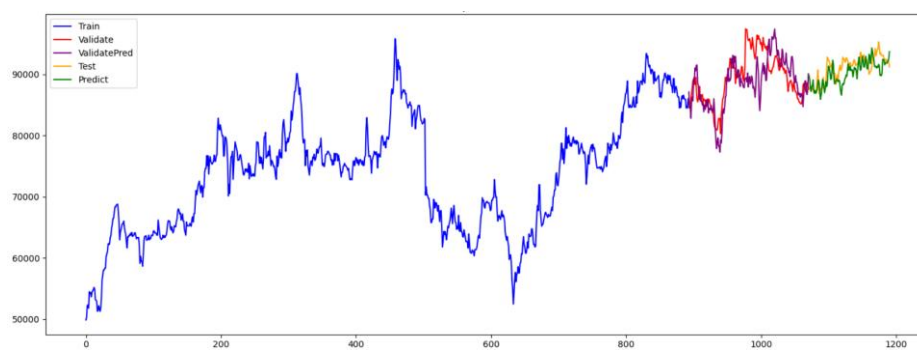


Fig. 16. SVR Forecast with a 75/15/10 Data Split



Fig. 17. LightGBM Forecast with a 75/15/10 Data Split

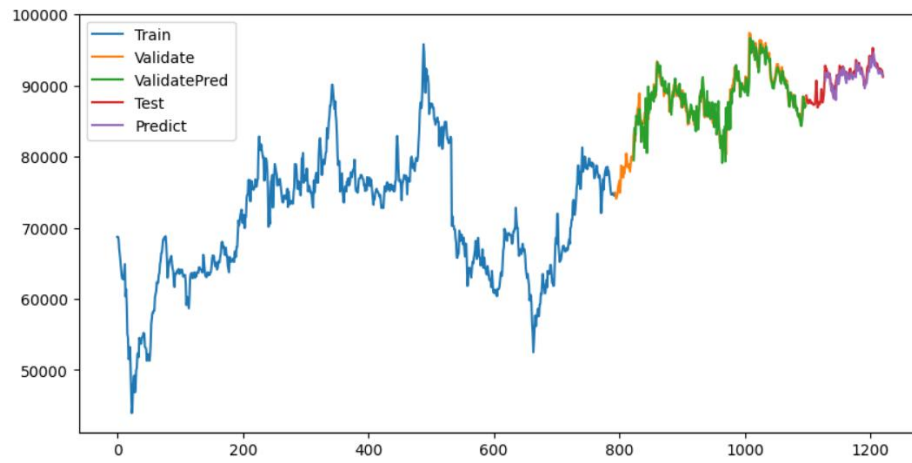


Fig. 18. RNN with a 65/25/10 Data Split

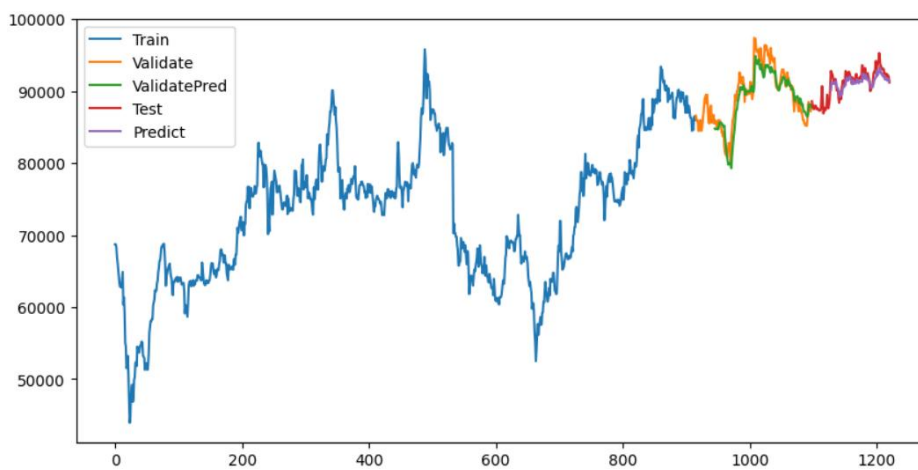


Fig. 19. LSTM Forecast with a 75/15/10 Data Split

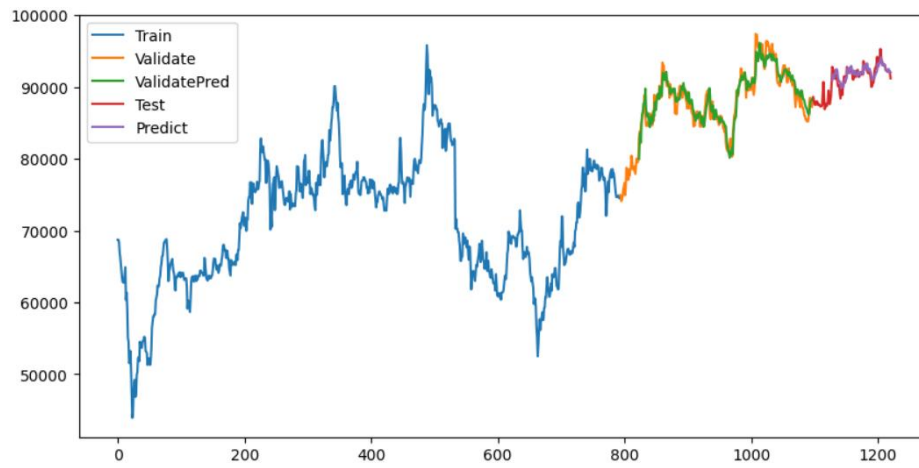


Fig. 20. GRU with a 65/25/10 Data Split



Fig. 21. CNN + LSTM Forecast with a 75/15/10 Data Split

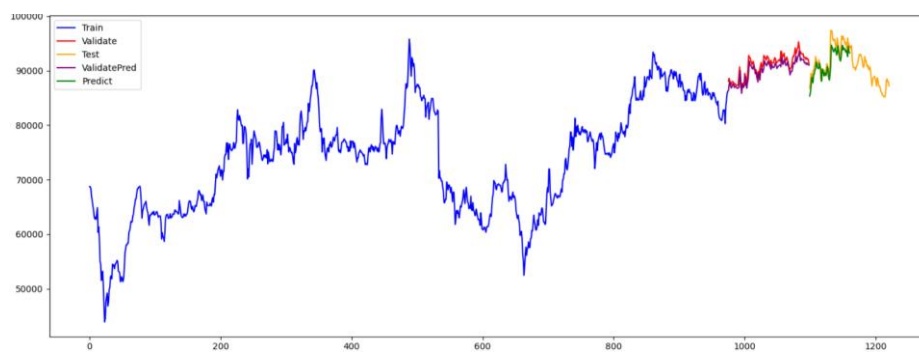


Fig. 22. CNN + Transformer Forecast with a 75/15/10 Data Split

Based on these results, Linear Regression and GRU were selected as the top two models for forecasting VCB stock prices over the next 30 days. Linear Regression offers superior accuracy combined with high interpretability, whereas GRU demonstrates robustness and strong generalization across various training configurations.



Fig. 23. 30-Day Forecast of VCB Stock Prices Using Linear Regression

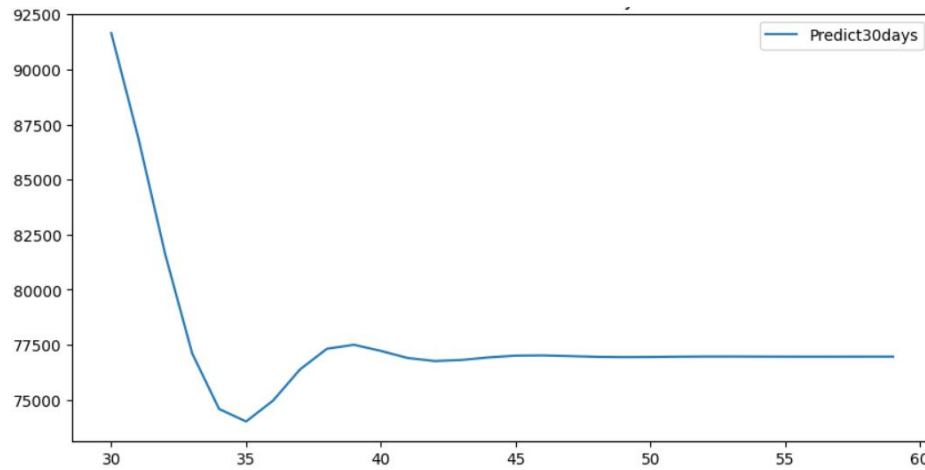


Fig. 24. 30-Day Forecast of VCB Stock Prices Using GRU

5 Conclusion

Challenges Encountered

The study encountered a few challenges, including limited dataset size and scope, which affected model generalization. The volatility and non-stationarity of stock prices also made consistent forecasting difficult. Deep learning models required intensive tuning and computational resources, while traditional models struggled to capture complex temporal patterns.

Summary of Results

This study compared ten models across three train-validation-test splits. Linear Regression consistently produced the lowest MAPE ($\approx 0.30\%$), demonstrating high accuracy despite its simplicity. GRU emerged as the most robust deep learning model with balanced error metrics. Based on overall performance, Linear Regression and GRU were selected as the top two models for forecasting VCB stock prices.

Future work

To enhance the effectiveness of stock price forecasting, subsequent studies may consider expanding the dataset with longer historical periods and incorporating supplementary financial indicators. Additionally, applying advanced deep learning or hybrid modeling techniques could further improve predictive accuracy. Strengthening model generalization and minimizing overfitting remain essential for developing robust and reliable forecasting systems applicable to real-world financial decision-making.

Acknowledgments.

The authors would like to express their profound gratitude to Mr. Duong Phi Long, M.Sc., Lecturer at the Faculty of Information Systems, University of Information Technology – Vietnam National University Ho Chi Minh City, for his valuable guidance, insightful feedback, and continuous encouragement throughout the duration of this research project. His deep expertise in data science and his dedicated mentorship were instrumental in shaping the direction and quality of this work.

This study also provided a meaningful opportunity for the research team to integrate academic knowledge with practical applications, particularly in the context of financial forecasting using machine learning and deep learning methodologies. It has significantly contributed to our professional growth, collaborative skills, and critical thinking in addressing real-world data challenges.

Despite our efforts to conduct the study with rigor and precision, we acknowledge that certain limitations remain, and we sincerely welcome constructive feedback to further refine and improve the research in future endeavors.

References

- [1] S. Vo, “Vietnam stock market and its development,” *Asian Journal of Finance & Accounting*, vol. 13, no. 1, pp. 1–18, 2021.
- [2] M. Fischer and K. Krauss, “Deep learning in stock price prediction,” *Journal of Forecasting*, vol. 39, no. 5, pp. 730–744, 2020.
- [3] A. Patel, H. Mehta, S. Patel, and A. Patel, “Predicting stock and stock price index movement using trend deterministic data preparation and machine learning techniques,” *Expert Systems with Applications*, vol. 42, no. 1, pp. 259–268, 2015.
- [4] G. James, D. Witten, T. Hastie, and R. Tibshirani, *An Introduction to Statistical Learning*, Springer, 2013. [Online]. Available: <https://www.statlearning.com/>
- [5] R. J. Hyndman and G. Athanasopoulos, *Forecasting: Principles and Practice*, 3rd ed., 2021. [Online]. Available: <https://otexts.com/fpp3/>

- [6] G. E. P. Box, G. M. Jenkins, G. C. Reinsel, and G. M. Ljung, *Time Series Analysis: Forecasting and Control*, 5th ed., Wiley, 2015. [Online]. Available: <https://www.wiley.com/en-us/Time+Series+Analysis%3A+Forecasting+and+Control,+5th+Edition-p-9781118675021>
- [7] R. J. Hyndman and G. Athanasopoulos, *Forecasting: Principles and Practice*, 3rd ed., 2021. [Online]. Available: <https://otexts.com/fpp3/dynamic.html>
- [8] G. E. P. Box, G. M. Jenkins, G. C. Reinsel, and G. M. Ljung, *Time Series Analysis: Forecasting and Control*, 5th ed., Wiley, 2015.
- [9] A. J. Smola and B. Schölkopf, “A tutorial on support vector regression,” *Statistics and Computing*, vol. 14, no. 3, pp. 199–222, 2004.
- [10] G. Ke et al., “LightGBM: A Highly Efficient Gradient Boosting Decision Tree,” in *Advances in Neural Information Processing Systems (NeurIPS)*, vol. 30, 2017.
- [11] J. L. Elman, “Finding structure in time,” *Cognitive Science*, vol. 14, no. 2, pp. 179–211, 1990.
- [12] S. Hochreiter and J. Schmidhuber, “Long short-term memory,” *Neural Computation*, vol. 9, no. 8, pp. 1735–1780, 1997.
- [13] K. Cho et al., “Learning phrase representations using RNN encoder–decoder for statistical machine translation,” *arXiv preprint, arXiv:1406.1078*, 2014.
- [14] J. Donahue et al., “Long-term Recurrent Convolutional Networks for Visual Recognition and Description,” in *Proc. IEEE Conf. on Computer Vision and Pattern Recognition (CVPR)*, 2015.
- [15] Z. Zeng et al., “Financial time series forecasting using CNN and Transformer,” *arXiv preprint, arXiv:2304.04912*, 2023.
- [16] Wikipedia contributors, “Linear regression,” Wikipedia, 2024. [Online]. Available: https://en.wikipedia.org/wiki/Linear_regression
- [17] Wikipedia contributors, “Mean squared error,” Wikipedia, 2024. [Online]. Available: https://en.wikipedia.org/wiki/Mean_squared_error
- [18] J. Brownlee, “Introduction to SARIMA for Time Series Forecasting in Python,” 2017. [Online]. Available: <https://machinelearningmastery.com/sarima-for-time-series-forecasting-in-python/>
- [19] J. Brownlee, “Introduction to SARIMAX for Time Series Forecasting in Python,” 2017. [Online]. Available: <https://machinelearningmastery.com/sarimax-for-time-series-forecasting-in-python/>
- [20] “SARIMAX Model,” *Statsmodels Documentation*. [Online]. Available: <https://www.statsmodels.org/stable/statespace.html#sarimax>
- [21] H. Drucker, C. J. C. Burges, L. Kaufman, A. Smola, and V. Vapnik, “Support Vector Regression Machines,” in *Advances in Neural Information Processing Systems (NeurIPS)*, vol. 9, 1997.
- [22] Microsoft, “LightGBM Documentation,” 2023. [Online]. Available: <https://lightgbm.readthedocs.io/>
- [23] Y. Bengio, P. Simard, and P. Frasconi, “Learning long-term dependencies with gradient descent is difficult,” *IEEE Transactions on Neural Networks*, vol. 5, no. 2, pp. 157–166, 1994.
- [24] K. Greff et al., “LSTM: A search space odyssey,” *IEEE Trans. on Neural Networks and Learning Systems*, vol. 28, no. 10, pp. 2222–2232, 2017.
- [25] J. Chung et al., “Empirical evaluation of gated recurrent neural networks on sequence modeling,” *arXiv preprint, arXiv:1412.3555*, 2014.
- [26] J. Y. H. Ng et al., “Beyond Short Snippets: Deep Networks for Video Classification,” in *Proc. IEEE Conf. on Computer Vision and Pattern Recognition (CVPR)*, 2015.
- [27] Y. LeCun, Y. Bengio, and G. Hinton, “Deep Learning,” *Nature*, vol. 521, no. 7553, pp. 436–444, 2015.

- [28] A. Vaswani et al., “Attention Is All You Need,” arXiv preprint, arXiv:1706.03762, 2017.
- [29] E. I. D. Team, “MAPE (Mean Absolute Percentage Error),” Oracle Help Center. [Online]. Available: https://docs.oracle.com/en/cloud/saas/planning-budgeting-cloud/pfusu/insights_metrics_MAPE.html
- [30] “RMSE: Root Mean Square Error,” Statistics How To. [Online]. Available: <https://www.statisticshowto.com/probability-and-statistics/regression-analysis/rmse-root-mean-square-error/>
- [31] Deepchecks, “What is Mean Absolute Error? Formula & Significance.” [Online]. Available: <https://www.deepchecks.com/glossary/mean-absolute-error/>